

Subject: Operating System

Unit : 3

Prof. Ashwani Mishra

Assistant Professor
AIML Department

CONTENTS

- Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection & Recovery

Deadlocks

- A process requests resources, and if the resources are not available at that time, the process enters a wait state.
- Sometimes, a waiting process is never able to change state; because the resources it has requested are held by another waiting process. This situation is called a Deadlock. This is an unwanted situation.

System Model

- A system consists of a finite number of resources to be distributed among a number of competing processes. The resources are partitioned into several types, each consisting of some number of identical instances.
- Memory space, CPU cycles, files and I/O devices (such as printers and DVD Drives) are examples of resource types. If a process requests an instance of resource type, the allocation of any instance of the type will satisfy the request. Under the normal mode of operation, a process may utilise a resource in the following sequence:
 - a. Request:** If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.
 - b. Use:** The process can operate on the resource.
 - c. Release:** The process releases the resource. The request and release are system calls. Examples are request () and release ().

Deadlock Characterization

A deadlock situation can arise if the following four conditions occur at the same time in a system.

Necessary conditions

- **Mutual Exclusion:** Resources are allocated to the process on a non-sharable basis. If another process requests for the resource held by another process, then the requesting process must be delayed until the requested resource is delayed.
- **Hold and Wait:** There must exist a process that is holding at least one resource and is waiting to acquire additional resources that are currently being held by other processes.
- **No Preemption:** Resources can not be preempted, i.e., a process can release resources voluntarily after the task has been done.
- **Circular Wait:** A set $\{P_0, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 and P_i is waiting for a resource held by P_{i+1} P_{n-1} waiting for resources held by P_n , and P_n is waiting for a resource held by P_0 .

A directed edge $P_i \rightarrow R_j$ is called a request edge; a directed edge $R_j \rightarrow P_i$ is called an assignment edge.

Methods for handling deadlocks

- Generally speaking, we can deal with the deadlock problem in one of three ways:
 - We can use a protocol to prevent or avoid deadlocks, ensuring that the system will *never* enter a deadlock state.
 - We can allow the system to enter a deadlock state, detect it, and recover.
 - We can ignore the problem altogether and pretend that deadlocks never occur in the system.
 - The third solution is the one used by most operating systems, including UNIX and Windows; it is then up to the application developer to write programs that handle deadlocks.
- Next, we elaborate briefly on each of the three methods for handling deadlocks. However, before proceeding, we should mention that some researchers have argued that none of the basic approaches alone is appropriate for the entire spectrum of resource allocation problems in operating systems.
- The basic approaches can be combined, however, allowing us to select an optimal approach for each class of resources in a system.
- To ensure that deadlocks never occur, the system can use either a deadlock-prevention or a deadlock-avoidance scheme. Deadlock prevention provides a set of methods for ensuring that at least one of the necessary conditions cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made.

Process Creation

- If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation may arise.
- In this environment, the system can provide an algorithm that examines the state of the system to determine whether a deadlock has occurred and an algorithm to recover from the deadlock (if a deadlock has indeed occurred).
- If a system neither ensures that a deadlock will never occur nor provides a mechanism for deadlock detection and recovery, then we may arrive at a situation where the system is in a deadlocked state yet has no way of recognising what has happened.
- In this case, the undetected deadlock will result in the deterioration of the system's performance, because resources are being held by processes that cannot run and because more and more processes, as they make requests for resources, will enter a deadlocked state. Eventually, the system will stop functioning and will need to be restarted manually.
- Although this method may not seem to be a viable approach to the deadlock problem, it is nevertheless used in most operating systems, as mentioned earlier. In many systems, deadlocks occur infrequently (say, -once per year); thus, this method is cheaper than the prevention, avoidance, or detection and recovery methods, which must be used constantly. Also, in some circumstances, a system is in a frozen state but not in a deadlocked state.

Deadlock Prevention

- **Mutual Exclusion**
- The mutual exclusion condition must hold for non-sharable resources. For example, a printer cannot be simultaneously shared by several processes. Sharable resources, in contrast, do not require mutually exclusive access and thus cannot be involved in a deadlock. Read only files are a good example of a sharable resource.
- **Hold and Wait**
- To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a process requests a resource, it does not hold any other resources. One protocol that can be used requires each process to request and be allocated all its resources before it begins execution. We can implement this provision by requiring that system calls requesting resources for a process precede all other system calls.

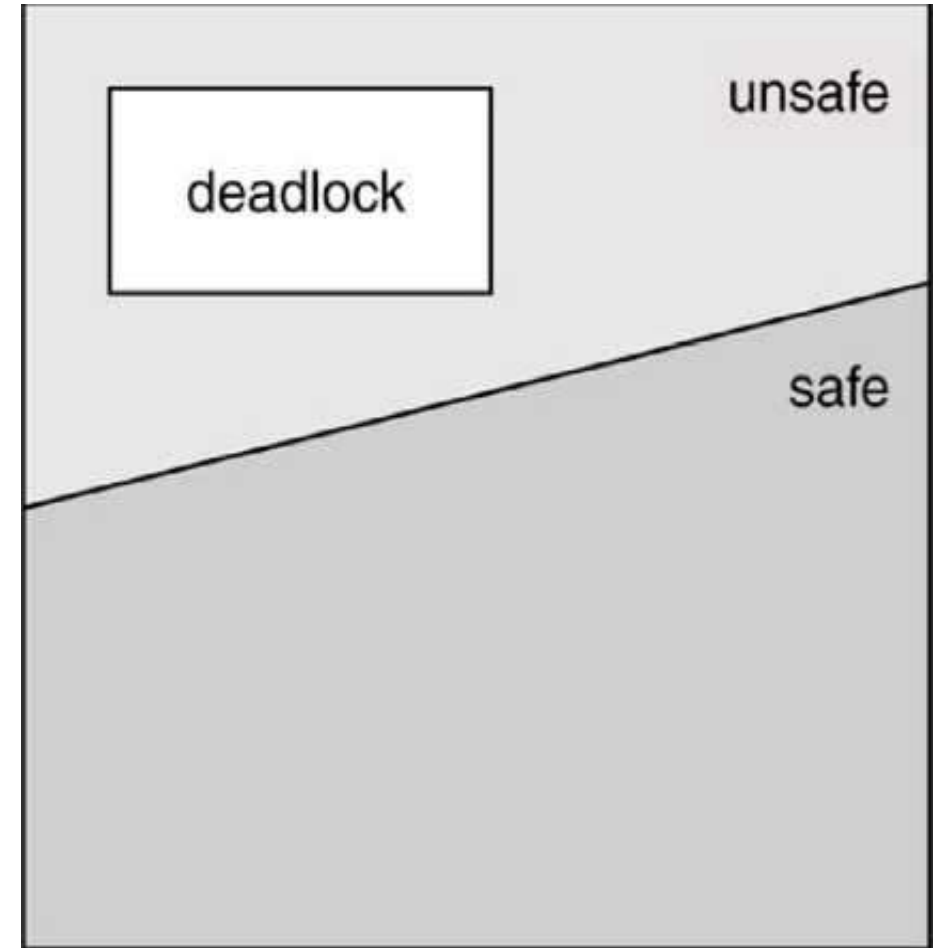
Continued...

- **No Preemption**
- The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated. To ensure that this condition does not hold, we can use the following protocol. If a process is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the process must wait), then all resources currently being held are preempted.
- **Circular Wait**
- The fourth and final condition for deadlocks is the circular-wait condition. One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

Deadlock Avoidance

State Spaces

- A safe state is not a deadlocked state.
- Conversely, a deadlocked state is an unsafe state.
- Not all unsafe states are deadlocks. An unsafe state may lead to a deadlock.
- As long as the state is safe, the operating system can avoid unsafe (and deadlocked) states.
- In an unsafe state, the operating system cannot prevent processes from requesting resources such that a deadlock occurs: The behaviour of the processes controls unsafe states.



Continued...

Safe, unsafe, deadlock

- To illustrate, we consider a system with 12 magnetic tape drives and three processes: P_0 , P_1 , and P_2 . Process P_0 requires 10 tape drives, process P_1 may need as many as 4 tape drives, and process P_2 may need up to 9 tape drives. Suppose that, at the time t_0 , process P_0 is holding 5 tape drives, process P_1 is holding 2 tape drives, and process P_2 is holding 2 tape drives. (Thus, there are 3 free tape drives.)

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	2

- At time t_0 , the system is in a safe state. The sequence $\langle P_1, P_0, P_2 \rangle$ satisfies the safety condition. Process P_1 can immediately be allocated all its tape drives and then return them (the system will then have 5 available tape drives); then process P_0 can get all its tape drives and return them (the system will then have 10 available tape drives); and finally, process P_2 can get all its tape drives and return them (the system will then have all 12 tape drives available).

Continued...

Deadlock Avoidance Algorithm (Banker's Algorithm):

- Deadlock can be avoided using the Banker algorithm. The idea behind is that only those processes whose requests (granted) does not result in a deadlock state, will be granted. If the allocation of resources to the process results in an unsafe state or deadlock state, the process is suspended until its request can be safely granted.
- For example, let us consider that there are 15 tape drives. Three processes A, B, C are allocated 4, 5, 3 tape drives respectively.
- Process A has 4 tape drives but require 14 tape drives.

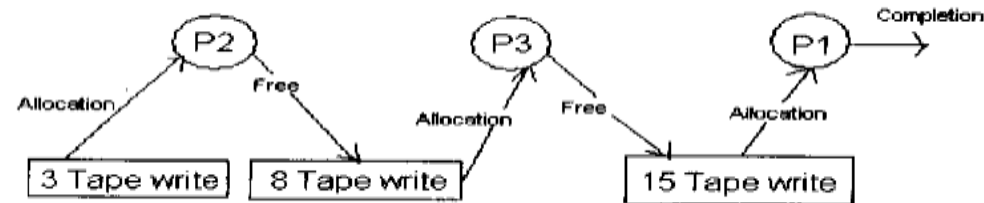
Safe State:

- Process B has 5 resources but require 8 to complete; Process C has 3 resources but require 7 to complete,
- A total of 12 processes are allocated to the processes.
- The number of available resources is $(15-12)$ 3 tape drives. The sequence (P2, P3, P1) does not result in deadlock.

Continued...

Process	Number of resources allocated	Maximum requirement of resources to complete	Maximum resources requested for completion
P1	14	14	$14-4=10$
P2	5	8	$8-5=3$
P3	3	7	$7-3=4$

Continued...



Unsafe State:

Process A has 4 tape drives but require 14 tape drives.

Process B has 5 tape drives but require 9 tape drives.

Process C has 3 tape drives but require 7 tape drives.

Process	Number of resources allocated	Maximum requirement of resources to complete	Maximum resources requested for completion
P1	4	14	$14-4=10$
P2	5	9	$9-5=4$
P3	3	7	$7-3=4$

Total allocated resources = 2

Maximum available resources $(15-12)=3$

None of the process requirements is less than or equal to 3. So each process enters into a deadlock state.

Continued...

Algorithm:

- If $\text{Request}_i \leq \text{Need}_i$, then Go To step 2 else, error.
- If $\text{Request}_i \leq \text{Available}$, then go to step 3 else, enter a wait state.
- The system assumes that the requested resources have been allocated to the process.
- Following actions are taken.
- $\text{Available} = \text{Available} - \text{Request}_i$
- $\text{Allocation}_i = \text{Allocation} + \text{Request}_i$
- $\text{Need}_i = \text{Need}_i - \text{Request}_i$
- Then, a safety algorithm is invoked to find whether the resulting is safe or unsafe. If unsafe, then the previous state of a system is restored and the process P_i is made to wait.

Deadlock Detection & Recovery

Deadlock Detection

- If the deadlock prevention techniques and the deadlock avoidance techniques are not used then a deadlock occurs in the system. The system must provide algorithms to detect the deadlock and to recover the system from deadlock.

Single instance of each resource type:

- If all resources have only a single instance, then we can define a deadlock detection algorithm, that uses a variant of resource allocation graph called as wait-for graph, which is obtained by removing the resource and collapsing the appropriate edges. A deadlock exists in the system, if there is a cycle in the wait-for graph.

Deadlock Detection & Recovery

Several instances of resource type:

This algorithm employs several time varying data structures that are similar to those used in Banker's algorithm. The deadlock detection algorithm is as follows:

- **Step 1:**
Initialize $work = Available$ $finish[i] = false$ for $i = 0, 1, 2, \dots$
- **Step 2:**
Find an i such that
 $finish[i] = false$ and $Request[i] \leq work$ If no such i exists go to step 4
- **Step 3:**
 $work = work + Allocation[i]$ $finish[i] = true$, goto step 2
- **Step 4:**
If $finish[i] = false$ for some i , $1 \leq i \leq n$, Then the system is in deadlock state.
If $finish[i] = false$ then the process P_i is deadlock.

Recovery From Deadlock:

- We can recover from deadlock using two methods:
- One is to abort one or more processes to break the circular wait.
- The second option is to preempt the resources from one or more of the deadlock processes.

Deadlock Detection & Recovery

Process termination:

- **Abort all deadlock processes:** In this method we can break all deadlock cycles by termination all the processes in the system. This method is very expensive, as the processes that have computed for a long time must also be terminated.
- **Abort one process at a time until the deadlock cycle is eliminated:** In this method, we will abort one process and invoke a deadlock detection algorithm. If there is a deadlock again, then we will abort another process and continue until there is no deadlock in the system.

Resource Preemption:

- We preempt some resources from processes and give them to some other processes until the deadlock cycle is broken.
- We must be careful about which processes and which resources are to be preempted.
- We must ensure that, one process resources must be preempted only for a finite number of times, otherwise, starvation of that process occurs.
- To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

Continued...

If preemption is required to deal with deadlocks, then three issues need to be addressed:

- **Selecting a victim** - As in process termination, we must determine the order of preemption to minimize cost. Cost factors may include such parameters as the number of resources a deadlocked process is holding and the amount of time the process has thus far consumed during its execution.
- **Rollback**- If we preempt a resource from a process, what should be done with that process? Clearly, it cannot continue with its normal execution; it is missing some needed resource. We must roll back the process to some safe state and restart it from that state.
- **Starvation**- In a system where victim selection is based primarily on cost factors, it may happen that the same process is always picked as a victim. As a result, this process never completes its designated task, a starvation situation that must be dealt with in any practical system. Clearly, we must ensure that a process can be picked as a victim only a (small) finite number of times. The most common solution is to include the number of rollbacks in the cost factor.

THANK YOU